

# SOLUÇÃO DEFINITIVA — USE MYSQL2

## DIRETO, PARE DE USAR DRIZZLE

*Diretriz Técnica para Correção de Erro Crítico de Persistência no Kanban*

08 de agosto de 2026

### 1. Por que nada funcionou até agora

Você tentou 5 soluções e todas falharam. Aqui está o motivo técnico real que está impedindo a gravação dos dados no banco de dados e a consequente exibição dos cards no Kanban:

1. **Drizzle .values()** gera INSERT com TODAS as 32 colunas — não tem como evitar, é o comportamento padrão do ORM.
2. **db.execute()** NÃO EXISTE no seu Drizzle MySQL — você mesmo descobriu isso na Tentativa 4.
3. **.returning()** e **.\$returningId()** não funcionam em MySQL, são exclusivos para PostgreSQL e SQLite.
4. Não adianta "limpar" o `insertData` — o Drizzle sempre expande o schema completo para a query final.

**PARE** de tentar fazer o Drizzle funcionar para este INSERT específico. Use **mysql2** diretamente. O driver já está instalado no seu projeto (é a dependência que o Drizzle usa por baixo). É só importar e usar.

### 2. A SOLUÇÃO — 3 ARQUIVOS PARA ALTERAR

#### 2.1. Arquivo 1: Crie um arquivo novo — `server/lib/db-raw.ts`

```
import mysql from 'mysql2/promise';

let pool: mysql.Pool | null = null;

export function getRawPool(): mysql.Pool {
  if (!pool) {
    const databaseUrl = process.env.DATABASE_URL;
    if (!databaseUrl) {
      throw new Error('DATABASE_URL não configurada');
    }
    pool = mysql.createPool({
      uri: databaseUrl,<br/>
```

```

        waitForConnections: true,<br/>
        connectionLimit: 10,<br/>
        queueLimit: 0,
    });
}
return pool;
}

```

## 2.2. Arquivo 2: Altere server/routers/logistica.ts — função .create()

Substitua COMPLETAMENTE a função **.create()** pelo código abaixo para garantir o bypass do ORM:

```

import { getRawPool } from '../lib/db-raw';

create: protectedProcedure
  .input(z.object({
    destinatarioNome: z.string(),<br/>
    destinatarioCnpj: z.string().optional(),<br/>
    cepDestino: z.string().optional(),<br/>
    municipio: z.string(),<br/>
    estado: z.string(),<br/>
    valorNf: z.string().optional(),<br/>
    observacoes: z.string().optional(),<br/>
    solicitanteNome: z.string().optional(),<br/>
    tipoMaterial: z.string().optional(),<br/>
    dimensoes: z.string().optional(),
  })))
  .mutation(async ({ input }) => {
    // 1. Extrair dimensões do JSON
    let largura: number | null = null;<br/>
    let altura: number | null = null;<br/>
    let comprimento: number | null = null;<br/>
    let pesoKg: number | null = null;

    if (input.dimensoes) {
      try {
        const volumes = JSON.parse(input.dimensoes);
        if (Array.isArray(volumes) && volumes.length > 0) {
          const v = volumes[0];
          largura = Number(v.largura) || null;
          altura = Number(v.altura) || null;
          comprimento = Number(v.comprimento) || null;
          pesoKg = volumes.reduce((acc: number, vol: any) => {
            return acc + (Number(vol.peso) || 0);
          }, 0);
        }
      } catch (e) {
        console.error('Erro ao parsear dimensoes:', e);
      }
    }
  })

```

```

}

// 2. Limpar CNPJ (só números)
const cnpjLimpo = input.destinatarioCnpj
  ? input.destinatarioCnpj.replace(/\D/g, '')
  : null;

// 3. INSERT direto com mysql2 – BYPASS do Drizzle
const pool = getRawPool();

try {
  const [result] = await pool.execute(
    `INSERT INTO cotacoes_frete (
      destinatarioNome,
      destinatarioCnpj,
      cepDestino,
      municipio,
      estado,
      dimensoesLargura,
      dimensoesAltura,
      dimensoesComprimento,
      pesoKg,
      valorNf,
      observacoes,
      solicitanteNome,
      tipoMaterial,
      status,
      temRetrabalho
    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
    [
      input.destinatarioNome,
      cnpjLimpo,
      input.cepDestino || null,
      input.municipio,
      input.estado,
      largura,
      altura,
      comprimento,
      pesoKg,
      input.valorNf ? parseFloat(input.valorNf) : 0,
      input.observacoes || null,
      input.solicitanteNome || null,
      input.tipoMaterial || null,
      'fila',
      false,
    ]
  );

  const insertId = (result as any).insertId;
  console.log('INSERT SUCESSO - ID:', insertId);<br/>
  return { success: true, id: insertId };<br/>
}

```

```

    } catch (error: any) {<br/>
      console.error('INSERT ERRO:', {<br/>
        message: error.message,<br/>
        code: error.code,<br/>
        errno: error.errno,<br/>
        sql: error.sql,
      });
      throw new TRPCError({
        code: 'INTERNAL_SERVER_ERROR',<br/>
        message: `Erro ao criar solicitação: ${error.message}`,
      });
    }
  }
},

```

### 2.3. Arquivo 3: Altere client/src/pages/logistica/NovaCotacaoDialog.tsx

```

const handleSubmit = () => {
  // Construir JSON de dimensões
  const dimensoes = volumes.map(v => ({
    largura: Number(v.largura) || 0,<br/>
    comprimento: Number(v.comprimento) || 0,<br/>
    altura: Number(v.altura) || 0,<br/>
    peso: Number(v.peso) || 0,
  }));

  create.mutate({
    destinatarioNome: form.destinatarioNome,<br/>
    destinatarioCnpj: form.cnpj || undefined,<br/>
    cepDestino: form.cepDestino || undefined,<br/>
    municipio: form.municipio,<br/>
    estado: form.estado,<br/>
    valorNf: form.valorNf || "0",<br/>
    observacoes: form.observacoes || undefined,<br/>
    solicitanteNome: solicitanteNome || "Usuário",<br/>
    tipoMaterial: "Letreiro / Sinalização",<br/>
    dimensoes: JSON.stringify(dimensoes),
  });
};

```

## 3. Por que isto vai funcionar (garantido)

1. **mysql2/promise** já está instalado — é a dependência base do seu projeto.
2. **pool.execute()** usa prepared statements reais do MySQL — elimina erros de sintaxe e tipos.
3. Você escreve o SQL manualmente com **APENAS 15 colunas** (não 32).
4. Os **?** são placeholders que tratam valores nulos e strings automaticamente.
5. **NÃO usa Drizzle** em momento algum para este INSERT crítico.

## 4. O QUE NÃO FAZER DE JEITO NENHUM

**Atenção:** Não tente adaptar o Drizzle nesta função. Siga as restrições abaixo:

- NÃO use `db.insert(cotacoesFrete).values()` — gera 32 colunas.
- NÃO use `db.execute()` — não existe no seu Drizzle MySQL.
- NÃO tente `db.run()` — também pode não existir.
- NÃO use `.returning()` ou `.$returningId()` — MySQL não suporta.
- NÃO inclua `pedidoCnpj`, `pedidoEndereco`, `pedidoCep` — não existem na tabela.
- NÃO use `as any` no mutate.

## 5. DEPOIS DE IMPLEMENTAR, TESTE ISTO:

1. Criar solicitação com OS 6906 (ANDRADINA) — deve ir para coluna "Fila" do Kanban.
2. Criar solicitação com G3 SOLUCOES VISUAIS (TAQUARITINGA) — deve ir para coluna "Fila".
3. Verificar no banco: `SELECT * FROM cotacoes_frete ORDER BY id DESC LIMIT 5`.
4. O Kanban deve mostrar os cards na coluna "Fila".
5. `destinatarioCnpj` deve ter apenas números (sem pontos/barras).

---

Local e data: Brasília, 08 de agosto de 2026

*Documento elaborado em 08 de agosto de 2026. As informações contidas são de responsabilidade do solicitante.*